

COMP 766: Assignment 1
Available: Friday, September 25th, 2026
Due Date: Friday, October 9th, 2026

This assignment considers the problem of curve inference, which we have been discussing in some length recently. Finger prints are a special class of challenging patterns for naive edge detection algorithms to analyze because the features have low contrast locally and they appear at very fine scales. Other more obvious challenging cases could be hair patterns. For this assignment we've provided an example of each for you to test your algorithms on. You can convert the originals to greyscale or just work with one of its 3 channels (red, green or blue). You are encouraged to use Matlab for your implementations, for ease of prototyping and to be able to generate visualizations with ease. But this is not a requirement. Feel free to use Python, and associated packages. You can use AI tools for assistance, preparation of figures, etc. If you do, please include an honest statement explaining how these tools were used. See the section titled **Notes** below. Feel free to share partial results with me on Ed->Discussion, via private posts, if you want feedback. Queries of a more general type, which relate clarification but don't give away an answer to a question, might benefit the entire class, so please make them via public posts on Ed.

1 Part I: (30%) INITIAL TANGENT ESTIMATES

Convolve the input image with the template discussed in Section IV of the Trace Inference paper, using the following 16 orientations: $n \times \pi/16, n = 0, 1, 2, 3, 4, 5, 6, 7, \dots, 15$. In other words, you will perform 16 separate convolutions. Treat the constants $A, B, C, \sigma_1, \sigma_2, \sigma_3$ as inputs and select appropriate values for them by trial and error.

One way to do this is to visualize the template so that the central bright line is roughly the thickness of the curve like features and it is extended in the vertical direction for an appropriate length (see Fig. 4 in the paper). Ofcourse, not all features have exactly the same thickness, so for a fixed set of choices for the constants you will be detecting patterns of a particular width. You might want to generate a visualization of your convolution filter as a surface plot (in Matlab you can use *mesh*, *surfz* or a related function).

Now, for a fixed set of parameters for your measurement model, normalize the sixteen outputs to get your assignment vector p_i at each location i in the image. Visualize these initial estimates by drawing vectors in the direction λ for those assignments for which $p_i(\lambda)$ is greater than some threshold, e.g., 0.15. You might want to overlay these vectors on the original input image. What you should see is a large set orientation tangent estimates, where a subset correspond to the local orientation of finger print curves, or the local orientation of the hair strands.

2 Part II: (40%) CO-CIRCULARITY SUPPORT

Now, using the notions of compatibility developed in Section V: A, B, C, D of the paper, estimate the support $s_i(\lambda)$ for orientation λ at location i . More specifically,

$$s_i(\lambda) = \max_{k=1, K} \sum_{j=1}^n \sum_{\lambda'=1}^m r_{ij}^{kk'}(\lambda, \lambda') p_j(\lambda'),$$

where

$$r_{ij}^{kk'}(\lambda, \lambda') = c_{ij}(\lambda, \lambda') K_{ij}^k(\lambda, \lambda') C_{ij}^{kk'}(\lambda, \lambda').$$

This notion of compatibility includes the ideas of co-circularity, curvature classes and consistency of curvatures, but **NOT** lateral maxima selection, developed in Section V of the paper. Your implementation of this support function will involve choices for several parameters, as well as a choice for the number of curvature classes to use. As well, it is reasonable to build the support function $s_i(\lambda)$ using a large enough but not infinite circular neighborhood of each pixel i . You can visualize your results by drawing vectors in the direction λ for those assignments for which $s_i(\lambda)$ is greater than some threshold. You should now expect to see some of the initial tangent estimates being suppressed, especially those which do not have local co-circularity support. Ofcourse, the use of curvature classes and consistency of curvatures in defining the compatibility function really only makes sense as the assignments are iteratively updated, but this will happen in Part III.

3 Part III: (30%) RELAXATION LABELING

Using the initial assignment given by Part I and the co-circularity support given by Part II, compute the average local support

$$A(p) = \sum_{i=1}^b s_i(\lambda) p_i(\lambda).$$

Now, apply a few iterations of relaxation labeling to converge to a consistent labeling. Use the algorithm developed in the relaxation labeling paper, along with the radial projection update rule described in Appendix A. Visualize your results in the same way as before. Surviving tangents should now correspond to “true” finger print curves. Also, pay attention to what happens near bifurcations and crossings. In principle, you should be able to pick these up as well.

Notes You will turn in a PDF, and also supporting code. I will not necessarily run or check the code. I will focus on the PDF so think of this as a mini-report where

you develop and present your approach, and show and discuss results. Be sure to explain subtleties of your implementation. Furthermore, there are many refinements developed in the article (such as in Section VI: B, D) that you should not necessarily worry about. I am looking for evidence that you understood most of the technical aspects of the framework and were able to use it, and that your report presents your ideas clearly and creatively. To this end you should turn in a brief (no more than 10 pages, including results, and likely shorter) report. In your report you should:

1. Summarize your implementations and present your results for each part.
2. Explain any simplifying assumptions you have made and their consequences.
3. Explain your choices for the various parameters.
4. Provide some critical discussion of your results as well as of the overall framework.
5. Include an honest statement of AI tools if you used them, and what they were used for.